Universidad Nacional de Costa Rica

Sede Regional Brunca

Campus Coto y Pérez Zeledón

EIF207 Estructuras de Datos
PROYECTO II (15 %)

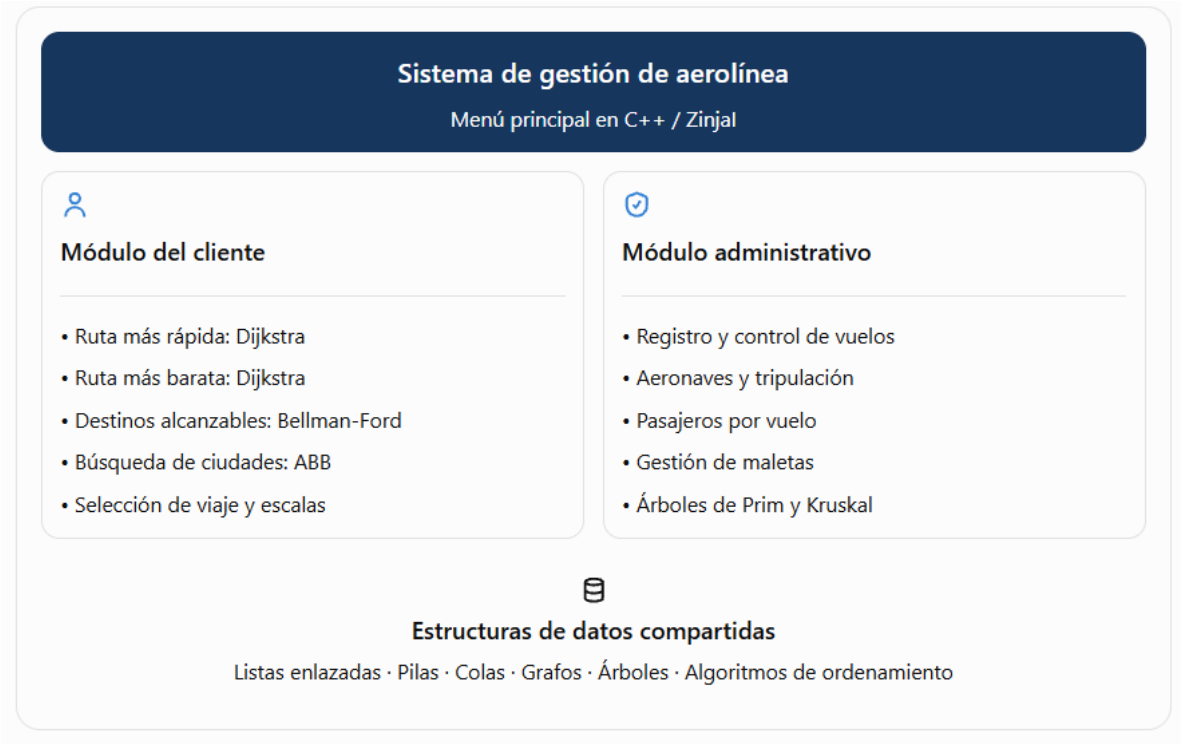
1. Descripción general

Usted debe implementar una plataforma de gestión de una aerolínea, debe construirla en C++, específicamente en Zinjai y utilizando exclusivamente las estructuras de datos estudiadas en este curso (tipos abstractos de datos, listas, pilas, colas, algoritmos de ordenamiento, grafos y árboles), así como los algoritmos vistos en clase.

El proyecto tendrá dos áreas principales:

- Módulo del cliente: consultar rutas, encontrar vuelos económicos o rápidos, explorar destinos y planificar viajes con escalas.
- Módulo administrativo: gestionar vuelos, aeronaves, tripulación, pasajeros y maletas, además de construir y recorrer los árboles solicitados.

La empresa cuenta con una serie de vuelos de diferentes destinos, para implementar los destinos usted debe utilizando estructuras de datos que le permitan administrar grafos como matrices de adyacencia o listas de adyacencia, para cada una de sus rutas tiene el costo y el tiempo para llegar de una localización a otra.



En el módulo del cliente existen varias herramientas en las que el cliente puede planificar sus viajes:

- El cliente puede escoger entre ir de un destino a otro y para esto debe define el destino donde se encuentra y selecciona el destino donde quiere ir, puede escoger entre seleccionar la búsqueda por tiempo o por costo, utilizando el algoritmo de Dijkstra debe recomendar el vuelo y mostrar todo el recorrido al cliente, indicando según corresponda el tiempo total del vuelo o el costo total del vuelo y todas las escalas que debe realizar (la ruta).
- El cliente puede estar planificando su viaje y no sabe a donde ir, para lo cual utiliza la opción que le permite calcular la ruta más corta de un punto dado a cualquier otra ciudad, para esto utiliza esta opción del algoritmo de Bellman-Ford, selecciona cual es el menor costo (puede ser tiempo o costo) para ir de la ciudad donde está a todas las demás ciudades donde la aerolínea tiene vuelos. El sistema debe mostrar todas las opciones posibles.
- Para buscar si una ciudad está en las rutas de la aerolínea se cuenta con un árbol de Búsqueda Binaria con el nombre de todas las ciudades visitadas. Muestre el árbol completo e integre un método para ver si una ciudad en particular esta en la ruta de la aerolínea.
- Una vez definido el cliente debe seleccionar el vuelo. Para poder conocer en que vuelos esta asignado un cliente debe contar con un árbol AVL, que le permita

mostrar fácilmente si un cliente este asignado al vuelo o no. Para la administración es importante poder contar con un despliegue total del árbol.

En el módulo administrativo, le empresa necesita contar con las siguientes herramientas:

- Control de vuelos: para poder contar con un control de vuelos total se debe controlar los siguientes puntos:
    - Aeronave: Registro completo de la aeronave, con detalle de placas, capacidad, marca, modelo, etc.
    - Ruta: Debe poder registrar cada una de las rutas de las aeronaves.
    - Tripulación: Debe poderse registrar la tripulación asignada a una aeronave.
    - Pasajeros: Se requiere contar con un detalle de los pasajeros que han seleccionado cada vuelo.
    - Maletas: las maletas siempre viajan por la ruta más barata, para cada pasajero en los vuelos por tiempo debe poderse considerar la ruta que llevara el equipaje.
- Construya con la información de los vuelos un árbol de expansión minimal utilizando el algoritmo de Prim ordenado por el costo del vuelo, debe mostrar el árbol de ciudades generado.
- Construya con la información de los vuelos un árbol de expansión minimal utilizando el algoritmo de Kruskal ordenado por la duración del vuelo, debe mostrar el árbol de ciudades generado.
- Implemente los métodos de recorrido de los árboles, utilícelos en cada árbol generado.

2. Diseño del grafo de vuelos

El grafo será la estructura más importante del proyecto.

1. Cada vértice representa una ciudad o aeropuerto.
2. Cada arista representa una ruta aérea directa entre dos ciudades.
3. Cada arista tendrá, como mínimo, un costo y una duración.
4. Si existen vuelos en ambos sentidos, se pueden registrar dos aristas dirigidas. No se debe asumir que el vuelo de regreso siempre tiene el mismo costo o duración.

3. Objetivos de aprendizaje

- Implementar y utilizar correctamente los árboles binarios de búsqueda.
- Implementar y utilizar correctamente las matrices de adyacencia para implementar grafos.
- Implementar y utilizar los algoritmos revisados en clase para: recorrer árboles, así como para cada operación de manejo de árboles.

4. Restricciones técnicas

Lenguaje: C++. Entorno / compilador: ZinjaI.

El proyecto es individual. Cada estudiante desarrolla, entrega y defiende su propia solución. No se permite usar std::stack, std::queue, std::deque, std::list, std::priority_queue ni std::sort para las estructuras y el ordenamiento que este enunciado pide implementar. Sí puede usar std::string, std::vector para colecciones auxiliares que no sean las estructuras evaluadas (por ejemplo, para leer un archivo), y la biblioteca estándar para entrada/salida y manejo de tiempo.

El estudiante debe estar en capacidad de explicar y defender cualquier parte del código durante la defensa oral. Cualquier indicio de código compartido entre estudiantes distintos, o de código no propio sin atribución, se tramita según el reglamento de fraude académico de la Universidad Nacional.

5. Interfaz y elementos gráficos (obligatorio)

La interfaz gráfica es obligatoria: debe usar SFML para dibujar las ciudades y rutas de la aerolínea. Debe incluir animación en las rutas de cada aerolínea.

6. Entregables

1. Código fuente completo, organizado en archivos .h/.cpp por módulo. Debe incluir un README que indique cómo compilar y ejecutar el programa y qué librería gráfica se usó.
2. Informe corto en PDF o Word (máximo 6 páginas) con: diagrama o descripción de cómo se relacionan las estructuras entre sí, complejidad teórica (notación asintótica) de cada operación implementada.
3. Un archivo comprimido (.zip) que agrupe todo lo anterior.

7. Tabla de calificación

El porcentaje final de este rubro corresponde al 15% de la nota del curso.

Importante: Si el proyecto no cumple las restricciones técnicas (sección 4), por ejemplo, utilizar estructuras prohibidas de la STL, no desarrollar en C++/Zinjal o entregar una aplicación únicamente en consola en lugar de la interfaz gráfica obligatoria, no se revisa y se califica con cero.

La calificación se compone de un puntaje del proyecto (0–100 puntos, apartados A, B y C), que posteriormente se multiplica por el factor de defensa (apartado D).

Nota final = Puntaje del proyecto X factor de defensa

Rubros del puntaje del proyecto (0–100)

Rubro	Valor
A. Documentación y calidad del desarrollo	15
Informe de análisis: descripción de estructuras, relación entre componentes, complejidad asintótica y justificación de los algoritmos utilizados.	6
Calidad del código: organización en archivos .h/.cpp, modularidad, legibilidad, comentarios relevantes, manejo adecuado de memoria y README con instrucciones de compilación y ejecución.	9
B. Planteamiento y solución adaptada (estructuras y algoritmos)	55
Grafo de vuelos propio: representación de ciudades, conexiones, costos y tiempos mediante matriz de adyacencia o listas de adyacencia.	10
Algoritmo de Dijkstra: búsqueda de la ruta más económica y la más rápida, mostrando recorrido, escalas y costo o duración total.	10
Algoritmo de Bellman-Ford: cálculo de rutas mínimas desde una ciudad de origen hacia todos los destinos alcanzables, según costo o tiempo.	8
Árbol binario de búsqueda (ABB): almacenamiento de ciudades, inserción, búsqueda, visualización y recorridos.	6
Árbol AVL: registro y búsqueda de pasajeros asignados a vuelos, balanceo, rotaciones y visualización del árbol.	9
Algoritmo de Prim: construcción del árbol de expansión mínima utilizando el costo de las rutas.	6
Algoritmo de Kruskal: construcción del árbol de expansión mínima utilizando la duración de las rutas.	6
C. Funcionamiento del aplicativo	30
Módulo administrativo: registro y gestión de aeronaves, vuelos, rutas, tripulación y pasajeros; consultas y asociaciones entre registros.	10

Módulo del cliente: consulta de destinos, selección de vuelos, planificación de viajes y presentación de alternativas de recorrido.	7
Gestión de equipaje: cálculo de la ruta más económica para las maletas y diferenciación respecto a la ruta seleccionada por tiempo.	5
Interfaz gráfica obligatoria (SFML): representación de ciudades y rutas, visualización de resultados de algoritmos, árboles y animación de recorridos.	8
SUBTOTAL — Puntaje del proyecto	100

El puntaje del proyecto se obtiene sumando los apartados A, B y C. Este subtotal (0–100) se multiplica posteriormente por el factor de defensa del apartado D.

D. Defensa del proyecto (factor multiplicador 0,00–1,00)

La defensa no suma puntos al proyecto: multiplica el puntaje obtenido en los apartados A, B y C.

Se evalúa individualmente mediante una sesión de preguntas del cuerpo docente sobre cualquier componente del entregable.

Criterio	Valor
D1. Dominio técnico: explica correctamente la implementación de grafos, árboles, algoritmos de búsqueda de rutas, árboles de expansión mínima y estructuras auxiliares. Demuestra comprensión del funcionamiento interno del código.	30
D2. Justificación de decisiones: argumenta por qué seleccionó cada estructura de datos, explica las diferencias entre Dijkstra y Bellman-Ford, entre Prim y Kruskal, y justifica las decisiones de diseño, eficiencia y complejidad algorítmica.	25
D3. Respuesta a preguntas imprevistas: modifica, depura o razona en vivo sobre su solución. Resuelve escenarios planteados por el docente, como agregar ciudades, modificar costos, eliminar rutas o explicar el comportamiento de un árbol ante nuevas inserciones.	25
D4. Comunicación: claridad al exponer, uso correcto de terminología técnica, explicación ordenada de los componentes, demostración del aplicativo y evidencia de comprensión del trabajo realizado.	20
TOTAL	100

Factor de defensa = Total obtenido / 100

Rúbrica de valoración del factor de defensa

Factor	Descriptor
0,00	No se presenta a la defensa o no logra explicar ningún componente del entregable.
0,01–0,40	Reconoce superficialmente el proyecto, pero no puede explicar cómo funcionan los algoritmos o las estructuras implementadas. Presenta respuestas memorizadas, incompletas o incorrectas.
0,41–0,60	Explica el funcionamiento general de la aplicación y reconoce las estructuras utilizadas, pero presenta dificultades para justificar decisiones algorítmicas, explicar el código o resolver variaciones del escenario.
0,61–0,80	Demuestra dominio de la mayoría de los componentes, explica correctamente los algoritmos principales y justifica las decisiones de implementación, aunque presenta dudas puntuales.
0,81–0,95	Domina el proyecto completo, explica las estructuras y algoritmos utilizados, justifica decisiones técnicas y responde con solvencia a preguntas imprevistas y modificaciones solicitadas.
0,96–1,00	Dominio total: comprende y explica cualquier componente del proyecto, razona sobre algoritmos y complejidades, realiza modificaciones en vivo y propone mejoras fundamentadas en eficiencia y diseño.

Condiciones de la defensa

1. La defensa es obligatoria. La ausencia injustificada implica un factor de 0,00, independientemente del puntaje obtenido en el entregable.
2. Si el estudiante no logra explicar el funcionamiento de un componente que aparece en su entrega (código, algoritmo, estructura, configuración o documento), el factor de defensa no podrá superar 0,50, independientemente de la calidad del aplicativo.
3. Se permite utilizar herramientas de asistencia, incluida la inteligencia artificial, durante el desarrollo, pero no durante la defensa. Se evalúa la comprensión del estudiante y su capacidad para explicar, justificar y modificar su propia solución.
4. El cuerpo docente podrá solicitar modificaciones en vivo como parte del criterio D3. Estas modificaciones podrán incluir cambios en rutas, costos, tiempos, ciudades, pasajeros, operaciones de árboles o comportamiento de los algoritmos.
5. El factor de defensa aplica únicamente sobre la nota del Proyecto II y no afecta los demás rubros de evaluación del curso.

Ejemplo: Un estudiante obtiene 90 puntos en el proyecto y un factor de defensa de 0,80.

90 X 0.80 = 72

Su calificación final del Proyecto II será de 72/100, equivalente a 10,8 puntos porcentuales de los 15% que representa el proyecto en el curso.

8. Entrega, defensa y logística

La entrega se realizará preferiblemente mediante el aula virtual institucional. El archivo comprimido deberá identificarse según el siguiente formato: Proyecto2_Nombre_Apellido_Cedula.zip. El archivo deberá contener el código fuente completo, organizado en archivos .h/.cpp, los recursos necesarios para compilar y ejecutar el aplicativo, un archivo README con instrucciones de ejecución y el informe técnico solicitado. El día establecido para la defensa, cada estudiante deberá presentar el sistema funcionando, demostrar los módulos de cliente y administración, ejecutar los algoritmos solicitados y explicar las estructuras de datos implementadas. El docente podrá seleccionar cualquier componente del proyecto para solicitar su explicación, ejecución, modificación o depuración. La presentación y defensa del proyecto son obligatorias para su evaluación.